

Trabajo Práctico N.º 2

Representación vectorial de texto: embeddings y búsqueda semántica

Gimbatti, Longo, Nasini, Paredes · Unidad 2

Corpus: 150 sinopsis del género Intriga, obtenidas en el TP1. Conjunto de evaluación propio: 12 consultas y 52 juicios de relevancia (`queries.json`), escritos antes de ejecutar ningún modelo.

1. Qué modelo elegiríamos para producción

SBERT (distiluse-base-multilingual-cased-v1), con una reserva importante.

precision@	Azar	TF-IDF	Prom. W2V (SBW)	SBERT
@5	0.029	0.250	0.283	0.333
@10	0.028	0.200	0.183	0.242

Gana en el promedio, 11× por encima del piso de azar. El resto de los criterios:

- **Costo.** Una consulta tarda 29 ms en CPU (mediana de 20 corridas); codificar el corpus completo, 7 s. No hace falta GPU. TF-IDF es dos órdenes de magnitud más barato y pierde solo 0.08 de `precision@5`.
- **Privacidad.** Corre localmente: ni el corpus ni las consultas salen de nuestra infraestructura. Una API de terceros habría implicado enviar el catálogo completo a un proveedor externo.
- **Reproducibilidad.** Fijamos semilla en todo lo estocástico. Las dos fuentes reales de irreproducibilidad son externas: que HuggingFace retire el modelo y que el SBW-vectors-300-min5 deje de estar disponible.
- **Dependencia de terceros.** Es el punto débil: `sentence-transformers` arrastra PyTorch y descarga el modelo en tiempo de ejecución. Para producción guardaríamos una copia propia y fijaríamos versiones.

La reserva. Nuestros datos indican que la respuesta no es un modelo sino dos: TF-IDF gana en consultas léxicas y SBERT es el único que funciona en las semánticas. Una búsqueda híbrida superaría a ambos. No la implementamos —elegimos chunking como parte avanzada— pero es lo que haríamos con más tiempo.

2. Cuánto mejor que TF-IDF, y dónde gana cada uno

SBERT supera a TF-IDF por 0.083 de `precision@5` (0.333 vs 0.250): un 33 % relativo. Ese número solo es poco informativo. Segmentado por tipo de consulta:

precision@5	n	Azar	TF-IDF	Prom. W2V	SBERT
Léxicas (solapamiento alto)	6	0.027	0.400	0.433	0.367
Mixtas	3	0.033	0.200	0.267	0.467
Semánticas (solapamiento 0)	3	0.031	0.000	0.000	0.133

Dónde pierde SBERT: en las consultas léxicas, TF-IDF y el promedio de Word2Vec le ganan. El caso extremo es q02 (“Hércules Poirot y Agatha Christie”), con 100 % de solapamiento: los nombres propios son la señal, y contar palabras es estrictamente mejor que interpretar. Lo mismo en `recall@10` sobre léxicas (TF-IDF 0.733, SBERT 0.658).

Dónde gana: en las tres consultas semánticas TF-IDF saca exactamente 0.000. No es mala suerte, es aritmética: puntúa por coincidencia de términos y esas consultas no comparten ninguna palabra con los documentos relevantes, así que el producto es cero por construcción. SBERT es el único que recupera algo (`recall@10` = 0.233).

SBERT no es “mejor”: es el único que cubre un caso que los otros no pueden cubrir. Si los usuarios escriben con el vocabulario del catálogo, TF-IDF alcanza y es más barato. Si escriben con sus propias palabras, TF-IDF devuelve cero.

3. Qué mide y qué no mide la métrica

`precision@k` mide qué fracción del top-k coincide con nuestros juicios de relevancia. **No mide** el orden dentro del top-k (acertar en el puesto 1 o en el 5 vale igual); no mide cobertura —agregamos `recall@10` por separado, y cambia conclusiones: el promedio de Word2Vec supera a TF-IDF en `precision@5` y queda por debajo en `precision@10`—; no

mide si nuestros relevantes son los correctos, ya que los definimos nosotros leyendo sinopsis **promocionales**, escritas para vender y no para describir; y no da significancia estadística: con 12 consultas y un solo anotador, la diferencia de 0.033 entre SBERT y el promedio de Word2Vec **no es una diferencia**.

Dos límites que la métrica no habría detectado nunca.

El promedio de Word2Vec tiene una similitud media de **0.895** entre pares de libros al azar (desvío 0.032, mínimo 0.718): no existe un par que el modelo considere distintos. Sus similitudes de 0.75 —que parecen altas— están por debajo del par promedio del corpus. Puede ordenar, pero sus valores absolutos no significan nada: no sirve para un umbral ni para una medida de confianza. La tabla de precision@k no muestra nada de esto.

A la inversa, la proyección 2D sugería que SBERT no capta los géneros. Pero PC1+PC2 retienen solo el **8.5 %** de la varianza. Medido en las 512 dimensiones con un test de permutación, tres de los cuatro subgéneros tienen cohesión interna significativa (Histórico y Policiaco $p < 0.001$; Psicológico $p = 0.046$); el cuarto no (Aventuras, $p = 0.517$), y con 7 libros no podemos distinguir si el modelo no lo capta o si falta potencia estadística. La estructura estaba: fallaba el gráfico.

4. Un caso concreto donde la búsqueda falló

Síntoma. La consulta “un detective investiga un asesinato en una casa de campo inglesa”, filtrada por género Histórico, devolvió una **tabla vacía**, pese a que el corpus tiene 19 libros de ese género. Sin filtro devolvía cinco resultados sensatos.

Hipótesis. HNSW es un índice aproximado: junta ef_search candidatos ordenados por similitud **sin mirar el género**, y Postgres aplica el filtro después. Si entre esos candidatos no hay ninguno del género pedido, no queda nada que devolver. Con ef_search = 40 por defecto y 150 documentos, el índice mira apenas un cuarto del corpus.

Verificación. Recall del índice frente a la búsqueda exacta:

filtro	libros	selectividad	ef = 40	ef = 100	ef = 400
Policiaco	39	26 %	1.00	1.00	1.00
Histórico	19	13 %	0.40	1.00	1.00
Psicológico	10	7 %	0.00	0.80	1.00
Aventuras	7	5 %	0.20	0.60	1.00

El recall se derrumba conforme el filtro se vuelve más selectivo, y subir ef_search lo recupera por completo. Lo peligroso es que el sistema **no avisa**: devuelve una lista corta o vacía, que parece una respuesta legítima. Subir ef_search arregla el problema visitando casi todo el corpus, es decir renunciando a la ventaja del índice; a escala real corresponde detectar los filtros muy selectivos y resolverlos con búsqueda exacta sobre el subconjunto filtrado, o mantener índices parciales por género.

Segundo fallo (parte avanzada). Elegimos chunking porque medimos que el **92 %** de los documentos se trunca y que SBERT descarta el **46.3 %** del corpus antes de vectorizar. La hipótesis era que trocear mejoraría el recall en los documentos truncados. **Empeoró**: recall@10 pasó de 0.569 a 0.478, y -0.092 justamente sobre los relevantes truncados. Diagnóstico: puntuar cada libro por su mejor pedazo premia la longitud —un documento con 9 pedazos tiene nueve oportunidades de puntuar alto por azar—. Lo confirmamos midiendo la correlación entre cantidad de pedazos y posición en el ranking: inexistente con el documento entero ($p = +0.111$, $p = 0.18$) y significativa con chunking ($p = -0.313$, $p < 0.001$). El problema no es trocear sino el max-pooling; promediar los pedazos o normalizar por su cantidad sería el siguiente experimento. No lo llevaríamos a producción.

Uso de asistentes de IA

Usamos Claude (Anthropic) a lo largo de todo el TP: para escribir el código del notebook y los scripts auxiliares, para armar el borrador del conjunto de evaluación a partir de la lectura del corpus, para diagnosticar errores de entorno y de SQL. Revisamos y ajustamos los juicios de relevancia de queries.json y verificamos cada conclusión contra las salidas del notebook. En varios puntos el asistente afirmó resultados que la ejecución desmintió —que el problema de recall de HNSW no se vería con 150 documentos; que las similitudes de TF-IDF en una consulta semántica serían cero en todo el corpus— y esas afirmaciones se corrigieron con los números medidos. Coincide con la advertencia del enunciado: los asistentes son buenos generando pipelines que corren y malos advirtiendo que una métrica no significa lo que parece.